



PRACTICAL: 04

Aim: To implement and find the Time analysis of factorial program using iterative and recursive method.

A) Iterative Method

Algorithm:

Input : A positive integer n.
Output : factorial of n.
step 1 : start
step 2 : Read n
step 3 : set fact = 1
step 4 : set i = 1
step 5 : Repeat while i ≤ n
 fact = fact × i
 i = i + 1
step 6 : Display fact
step 7 : stop.

Program / Code:

```
#include <iostream>
using namespace std;
class Factorial {
public:
    int fact(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result = result * i;
        }
        return result;
    }
};
```

```
int main() {
    int n, result;

    Factorial obj;

    cout << "Enter the value to find its factorial: ";

    cin >> n;

    result = obj.fact(n);

    cout << "Using Iterative Method!" << endl;

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:

```

C:\Users\karth\Downloads\D > Enter the value to find its factorial: 8
Using Iterative Method!
Factorial of 8 is 40320

Process returned 0 (0x0)    execution time : 7.026 s
Press any key to continue.

```

Time Complexity:

ne complexity

$(i=1; i \leq n; i++) \{$
 fact = fact * i;
}

$i=1 \rightarrow 1 \cdot 1$ (no work)
 $i \leq n \rightarrow n+1$
 $i++ \rightarrow n$

fact = fact * i $\rightarrow n$

$T(n) = 1 + (n+1) + n \cdot n$
 $= 3n+2$
 $= O(n) \rightarrow \text{worst case}$

Best case
if $n=1$
 $T(n) = 1 + 1 + 1$
 $= 3$
 $\geq O(1)$

avg case
 $i \rightarrow 1$
 $n \rightarrow n/2 + 1$
 $n \rightarrow n/2$
fact = fact * i $\rightarrow n/2$
 $T(n) = 1 + (n/2 + 1) + n/2 \cdot n/2$
 $= 3 \cdot \frac{n}{2} + 2$
 $= O(n)$



B) Recursive Method:

Algorithm:

Algo:
Input : A positive integer n .
Output : factorial of n .
step 1 : start
step 2 : Read n .
step 3 : If $n=0$ or $n=1$
 return.
step 4 : otherwise.
 return $n \times \text{factorial}(n-1)$
step 5 : Display result.
step 6 : stop

Program / Code:

```
#include <iostream>
using namespace std;
class Factorial {
public:
    int factr(int n) {
        if (n == 0 || n == 1) {
            return 1;
        }
        else {
            int result = n * factr(n - 1);
            return result;
        }
    }
};
```



```
    }  
};  
int main() {  
    int n, result;  
    Factorial obj;  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
    result = obj.factr(n);  
    cout << "Using Recursive Method!" << endl;  
    cout << "Factorial of " << n << " is " << result << endl;  
    return 0;  
}
```

Output:

Time Complexity:

Time complexity:

factorial (n)

{ if (n == 0 || n == 1) → 1
return n; → 1
return n * factorial(n-1); → T(n-1)

2

$T(n) = T(n-1) + 2$

$T(n) = T(n-2) + 2 + 2$

$T(n) = T(n-3) + 2 + 2 + 2$

⋮

$T(1) + 2n$

$= O(n) \rightarrow$ worst case.

Best case
when $n = 0$ or $n = 1$;
 $T(n) = 1$
 $= O(1)$

Average case
 $T(n) = T(n-1) + 2$
 $= O(n)$

Conclusion:

In this practical, we studied and implemented the factorial using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.